

Memo Recap Stage Python Intermédiaire

Programmation orientée objet

Classes

Objets

Pygame

Fenêtre de jeu

Événements

Charger et afficher des images

Dessiner des formes géométriques

Vérifier les collisions entre deux objets

Programmation orientée objet

Pour l'instant les seules façons de stocker des données pour notre programme que nous avons vu étaient les variables et les listes.

Prenons l'exemple d'un jeu vidéo RPG, où nous aimerions avoir deux héros qui possèdent plusieurs caractéristiques (points de vie, points de mana, force, armure, intelligence, etc.). Nous pourrions faire cela avec des variables:

```
pv_joueur1 = 100
pm_joueur1 = 100
agilite_joueur1 = 30
force_joueur1 = 10

pv_joueur2 = 10
pm_joueur2 = 100
agilite_joueur2 = 30
force_joueur2 = 10
```

Le problème de cette approche est que le programme ne sait pas que tous les variables contenant `joueur1` dans leur nom sont liées d'une certaine manière et que pour chaque héros que nous voulons créer dans notre programme nous devons recopier les lignes de chaque caractéristique.

C'est ici que la programmation orientée objet (POO) rentre en jeu.

Classes

La POO va nous permettre de créer un modèle - appelé `class` - à partir duquel nous allons pouvoir créer des objets de cette classe.

Une classe peut contenir deux choses:

- Les `attributs` que peut avoir un objet d'une classe (dans notre exemple les points des vies, armure etc.
- Les `méthodes` qui sont des fonctions qu'un objet d'une classe peut utiliser et ils sont définis dans la classe (dans notre exemple les héros pourront attaquer d'autres héros donc ils auront une méthode `attaquer`)

Nous allons définir une classe qui servira de modèle pour les héros que nous voulons créer dans notre jeu.

```
class Hero():
    def __init__(self, nom, pv, pm, armure, endurance, force, agilité, intelligence):
        self.nom = nom
        self.pv = pv
        self.pm = pm
        self.armure = armure
        self.endurance = endurance
        self.force = force
        self.agilité = agilité
        self.intelligence = intelligence

    def attaquer(self, ennemi):
        puissance_dattaque = self.force * 5
        degats = puissance_dattaque - ennemi.armure * 0.5
        print(self.nom, "attaque", ennemi.nom)
        ennemi.pv = ennemi.pv - degats
        print("il reste", ennemi.pv, "points de vie a", ennemi.nom)
```

Objets

La fonction `init` d'une classe sert à créer un objet de cette classe et définit les différents `attributs` qu'un objet de cette classe peut avoir, on appelle cette fonction un constructeur car elle nous permet de 'construire' ou créer un objet de cette classe.

Comme expliqué précédemment, une classe est un modèle pour un objet; il ne suffit pas de créer la classe, il faut utiliser son constructeur pour créer un objet de cette classe:

```
hero1 = Hero("Eric", 100, 100, 50, 20, 10, 10, 10)
```

Un des plus gros avantages d'utiliser la POO est le fait que tous les attributs du `hero1` que nous avons créé sont liés à ce héros. Ainsi nous pouvons accéder à ces attributs de la manière suivante:

```
print(hero1.nom)
#Eric
print(hero1.pv)
#100
```

Nous pouvons aussi créer autant d'objets que nous voulons à partir de cette classe et chaque objet aura ses propres caractéristiques ou attributs que nous pourrions manipuler et obtenir comme dans l'exemple ci-dessous:

```
hero2 = Hero("Luc", 100, 100, 50, 20, 10, 10, 10)

hero2.pv -= 50

print(hero2.pv)
#50
```

Chaque objet peut aussi 'utiliser' les fonctions ou méthodes définies par sa classe. Dans notre exemple, chaque héros (`hero1`, `hero2`) peut utiliser la fonction `attaquer` pour attaquer un autre héros. Ici, le `hero1` attaque `hero2`:

```
hero1.attaquer(hero2)
#Eric attaque Luc
#Il reste 75.0 points de vie à Luc
```

Pygame

Fenêtre de jeu

Pour créer une fenêtre dans pygame, vous devez d'abord initialiser pygame avec `pygame.init()`. Ensuite, vous pouvez créer une fenêtre avec `pygame.display.set_mode()` en passant une taille de fenêtre en pixels.

```
import pygame

pygame.init()
fenetre = pygame.display.set_mode((800, 600))
```

Événements

Pour vérifier si des touches du clavier sont enfoncées dans pygame, vous pouvez utiliser la fonction `pygame.key.get_pressed()`.

Pour vérifier si une touche est enfoncée:

```
keys = pygame.key.get_pressed()
if keys[pygame.K_LEFT]:
    # La touche de direction gauche est enfoncée
if keys[pygame.K_RIGHT]:
    # La touche de direction droite est enfoncée
```

Charger et afficher des images

Pour charger une image dans pygame, vous pouvez utiliser la fonction

`pygame.image.load()`:

```
image = pygame.image.load('image.png')
```

Pour “coller” ou afficher cette image dans la fenêtre de jeu, utiliser la fonction `blit` de l’objet `fenetre` en passant en argument l’image suivi de sa position (x,y):

```
fenetre.blit(image, (0,0))
```

Dessiner des formes géométriques

Pour dessiner un rectangle, utiliser la fonction `pygame.draw.rect()` en passant en argument la surface sur laquelle dessiner le rectangle, la couleur du rectangle, les coordonnées du coin supérieur gauche, la largeur et la hauteur du rectangle:

```
# dessiner un rectangle rouge de 100x100 pixels en haut à gauche de la fenêtre

pygame.draw.rect(fenetre, (255, 0, 0), (0, 0, 100, 100))
```

Vérifier les collisions entre deux objets

Pour vérifier les collisions entre deux rectangles par exemple, utiliser la fonction `colliderect` de pygame:

```
rect1 = pygame.Rect(0, 0, 50, 50)
rect2 = pygame.Rect(25, 25, 50, 50)

if rect1.colliderect(rect2):
    print("Collision détectée")
```